iOS Transaction Services Guide

The Transaction Services Guide offers a comprehensive overview of the various merchant transaction services supported by the POS SDK.

Below are the key utility features available through the Transaction Service classes.

- **Sending Transaction Receipts**: To support digital confirmations, you can send transaction receipts via email or SMS using the Transaction Receipt Service. This enables customers to receive proof of payment through their preferred communication channel.
- **Retrieving Transaction Summary**: Use the Transaction Summary Service to access summary data such as total amounts, transaction counts, and categorized breakdowns—helpful for evaluating overall sales performance.
- **Retrieving Transaction History**: The Transaction History Service allows merchants to view past sales by applying various search filters, with support for pagination to navigate large datasets efficiently.
- **Retrieving Transaction Details**: With the Transaction Detail Service, you can access comprehensive information for any specific transaction, including prices, taxes, discounts, tips, and items sold.

Prerequisites

Before you begin integrating the Deluxe POS SDKs, ensure you have the following prerequisites ready:

- **Merchant Client Application Credentials and Access Token:** Ensure that merchant onboarding is complete and that you've received your client application credentials and access token before proceeding.
- **Access to Deluxe Developer Portal:** This portal provides additional resources, API documentation, and integration

ON THIS PAGE

Prerequisites

Sending Transaction Receipts

Retrieving Transaction Summary

Retrieving Transaction History

Retrieving Transaction Detail

retrieve associated merchant accounts, which are necessary to establish a connection with the terminal.
- iOS Merchant Authorization & Accounts Guide

Note: *If you don't have a Deluxe Merchant Account or Access Token, simply visit Merchant Services Get Started and contact us today.*

Sending Transaction Receipts

Transaction Receipt service sends transaction receipts via email or SMS using the provided transaction ID. It supports seamless, digital delivery of payment confirmations to customers.

Step 1: Create a TransactionReceiptService Object

```
let receiptService = TransactionReceiptService()
```

Step 2: Create a SendTransactionReceiptRequest object

Create a SendTransactionReceiptRequest with the required transactionID, and either an emailTo address or an smsTo number with smsCode. If sending via email, smsTo and smsCode can be null; if sending via SMS, emailTo can be null.

```
let request = SendTransactionReceiptRequest(
 transactionID: "TXN1234567890",
 emailTo: "customer@example.com",
 smsTo: "",
 smsCode: ""
)
```

Step 3: Send Transaction Receipt

Pass the request and bearer token to the sendTransactionReceipt() method.

```
receiptService.sendTransactionReceipt(request:
request, bearerToken: bearerToken) { response,
error in
 if let result = response {

 } else if let error = error {

 }
}
```

Step 4: Response Handling

On success, sendTransactionReceipt() method returns a SendTransactionReceiptResponse object which contains following:

- **If success = true**, the receipt was delivered successfully, and the response includes a message confirming delivery via email or SMS.
- **If success = false**, the receipt delivery failed, and the reason can be found in the errorCode and message fields.

On failure, the method returns an error indicating a service fault.

Retrieving Transaction Summary

The Transaction Summary Service retrieves total transaction amounts, counts, and breakdowns by status or payment type.

Step 1: Create a TransactionSummaryService Object

```
let transactionSummaryService =
TransactionSummaryService()
```

Step 2: Create a GetTransactionSummaryRequest object

Create a GetTransactionSummaryRequest with the necessary parameters: dateFrom and dateTo to define the report's date range (should be in YYYY-MM-DD format), along with paymentType to filter by payment method, and transactionStatus to specify which transaction statuses to include.

```
let request = GetTransactionSummaryRequest(
 userId: "12345",
 accessToken: "761f497a-65c4-489e-973d-
79ab84fd3011",
 dateFrom: "2024-01-15",
 dateTo: "2024-01-31",
 paymentType: "Credit",
 transactionStatus: "Approved"
)
```

accessToken: This should be the access token for the merchant account selected by the user. Available merchant accounts can be retrieved using the MerchantAccountService.

getTransactionSummary() method.

```
transactionSummaryService.getTransactionSummary(req
uest: request, bearerToken: bearerToken) {
response, error in
 if let summary = response {

 } else if let error = error {

 }
}
```

The **bearerToken** should be the token obtained from the MerchantAuthorizationService.

Step 4: Response Handling

On success, the getTransactionSummary() method returns a GetTransactionSummaryResponse object containing the following details:

- **If success = true**, the response includes TransactionSummary objects representing the merchant's transaction details.
- **If success = false**, the failure reason is provided in the errorCode and message fields.

On failure, the method returns an error object that describes the cause of the failure.

Retrieving Transaction History

Transaction History service retrieves a paginated list of transactions within a specified date range. It provides structured access to individual transaction records and totals.

Step 1: Create a TransactionService Object

```
let transactionService = TransactionService()
```

Step 2: Create a TransactionHistoryRequest object

To create a TransactionHistoryRequest, specify the queryStartDate and queryEndDate in YYYY-MM-DD format to define the date range, use page and pageSize for pagination,

```
let request = TransactionHistoryRequest(
 userId: "",
 queryStartDate: "2024-01-01",
 queryEndDate: "2024-01-31",
 page: 1,
 pageSize: 10,
 accessToken: "761f497a-65c4-489e-973d-
79ab84fd3011",
 searchString: "invoice123"
)
```

accessToken: The access token associated with the merchant account selected by the user.

Step 3: Retrieve Transaction History

Pass the request and bearer token to the getTransactionHistory() method.

```
transactionService.getTransactionHistory(request:
request, bearerToken: bearerToken) { response,
error in
 if let result = response {

 } else if let error = error {

 }
}
```

The **bearerToken** should be the token received from the MerchantAuthorizationService.

Step 4: Response Handling

On success, getTransactionHistory() method returns a TransactionHistoryResponse object which contains the following:

- **If success = true,** the response includes a list of TransactionDetail objects representing the merchant's transaction details.
- **If success = false**, the failure reason is provided in the errorCode and message fields.

On failure, the method returns an error object that explains the reason for the failure.

Retrieving Transaction Detail

Step 1: Create a TransactionDetailService Object

```
let detailService = TransactionDetailService()
```

Step 2: Create a TransactionDetailRequest object

Create a TransactionDetailRequest with the necessary parameter transactionId, which is the unique identifier of the transaction for which the receipt is being requested.

```
let request = TransactionDetailRequest(
 transactionId: "TXN1234567890",
 userId: nil,
 accessToken: "761f497a-65c4-489e-973d-
79ab84fd3011"
```

accessToken: The access token associated with the merchant account selected by the user.

Step 3: Retrieve Transaction Details

Pass the request and bearer token to the getTransactionDetail() method.

```
detailService.getTransactionDetail(request:
request, bearerToken: bearerToken) { response,
error in
 if let detail = response {
 } else if let error = error {

 }
}
```

The **bearerToken** should be the token received from the MerchantAuthorizationService.

Step 4: Response Handling

On success, getTransactionDetail() method returns a TransactionDetailResponse object which contains the following:

- **If success = true**, response contains TransactionDetail indicating the details of the transaction.
- **If success = false**, indicates failure and corresponding failures reasons can be found in errorCode& message.

Note: *Please refer to the Complete SDK Reference for more details: iOS SDK Reference Guide*

Deluxe.com | Privacy Policy | Do Not Sell My Personal Information

Deluxe Corporation, 801 S Marquette Ave, Minneapolis, MN 55402.

© 2026 Deluxe Corporation